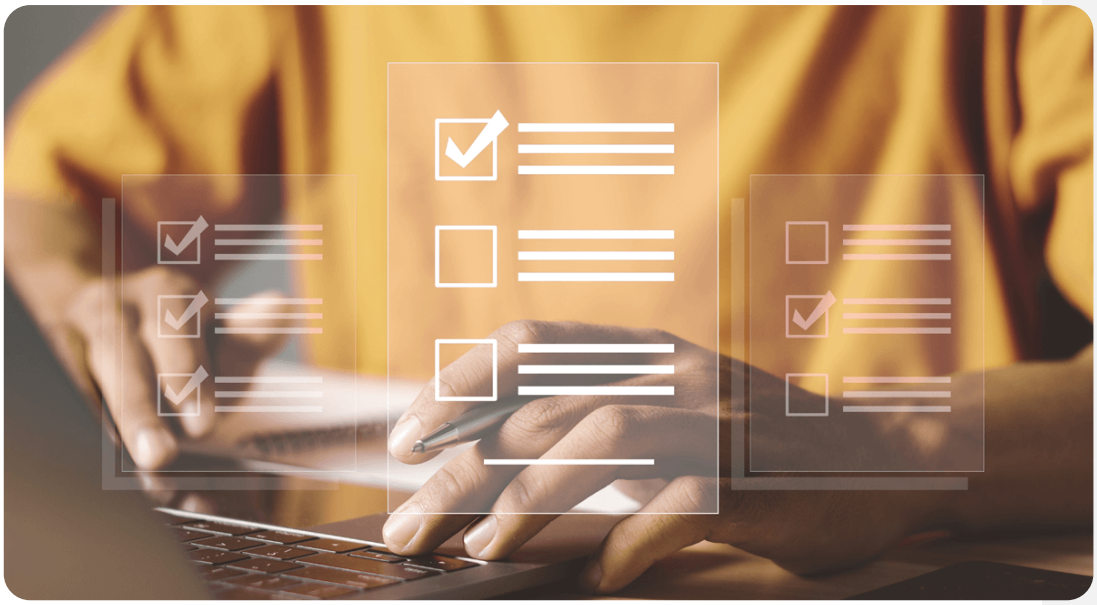
DuocUC | ONLINE



Semana 2
Desarrollo Backend III
(PBY2203)

Nombre estudiante: Raúl Zúñiga – Francisco Henríquez	
Asignatura: Desarrollo Backend III	**Carrera: Desarrollo de Aplicaciones**
Profesor: Gabriel Grobier	**Fecha:24-08-2026**

Descripción de la actividad

En esta actividad, implementarás un sistema de migración de procesos batch utilizando Spring Batch, con el objetivo de modernizar el sistema legacy del Banco XYZ. El enfoque será recrear tres procesos clave: el reporte de transacciones diarias, el cálculo de intereses mensuales y la generación de estados de cuenta anuales. El proyecto debe cumplir con los requisitos para garantizar la integridad y consistencia de los datos, aplicando buenas prácticas en la gestión de errores y optimización del rendimiento.

Instrucciones específicas

En función de la actividad formativa de la semana 1 llamada Analizando la arquitectura batch para procesar datos, deberás avanzar en el proyecto en su continuidad, realizando:

1. Configurar un Proyecto Spring Batch

Inicia un proyecto utilizando Spring Batch y configura Jobs y Steps para leer, procesar y escribir los datos de los archivos CSV. Versiónalo en el repositorio de proyectos de GitHub bajo una cuenta de tu propiedad.

2. Implementar un Procesamiento de Datos

Lee los archivos CSV. Aplica transformaciones y validaciones con un ItemProcessor para corregir o manejar los errores en los datos.

Escribe los datos procesados en una base de datos relacional. Puedes elegir entre PostgreSQL, MySQL u Oracle.

3. Manejo de Errores y Excepciones:

Deberás tener en cuenta que la data a ser procesada podría contener datos incorrectos y mal clasificados, por ello deberás manejar reglas para asegurar su consistencia.

4. Implementar políticas personalizadas y tolerancia a fallos:

Comentado [ve1]: Esto es lo que se le pide en la semana 1? si es así, tal vez la instrucción es sacarlo y dejar un texto que señale que en la continuidad de la actividad de la semana anterior, deberán realizar...

Comentado [AI2R1]: Preferiria dejarlo porque siempre es un tema conectar diferentes documentos para comprender el entregable completo. Aunque sea muy obvio, prefiero hacerlo asi. No se si estaran de acuerdo conmigo el resto pero en ese sentido por eso repeti esta parte del documento.
@Ignacio javier Villarroel Sanchez @Claudio Olavarria P.

Deberás tener en cuenta que deberás considerar políticas personalizadas para poder implementar una correcta tolerancia a fallos.

5. Implementar políticas de escalamiento:

Deberás tener en cuenta que deberás considerar políticas de escalamiento para poder utilizar 3 hilos de ejecución paralela con chunks de tamaño 5.

Requerimientos del Sistema

Basándote en los datos de https://github.com/KariVillagran/bank_legacy_data deberás reescribir los procesos en Spring Batch.

Configura un Job para cada uno de los tres procesos:

- **Reporte de Transacciones Diarias:** Procesar transacciones diarias para detectar anomalías y generar un resumen.
- **Cálculo de Intereses Mensuales:** Aplicar intereses sobre cuentas de ahorro y préstamos, y actualizar el saldo final en una base de datos.
- **Generación de Estados de Cuenta Anuales:** Compilar datos anuales para cada cuenta y generar un informe detallado para auditorías.

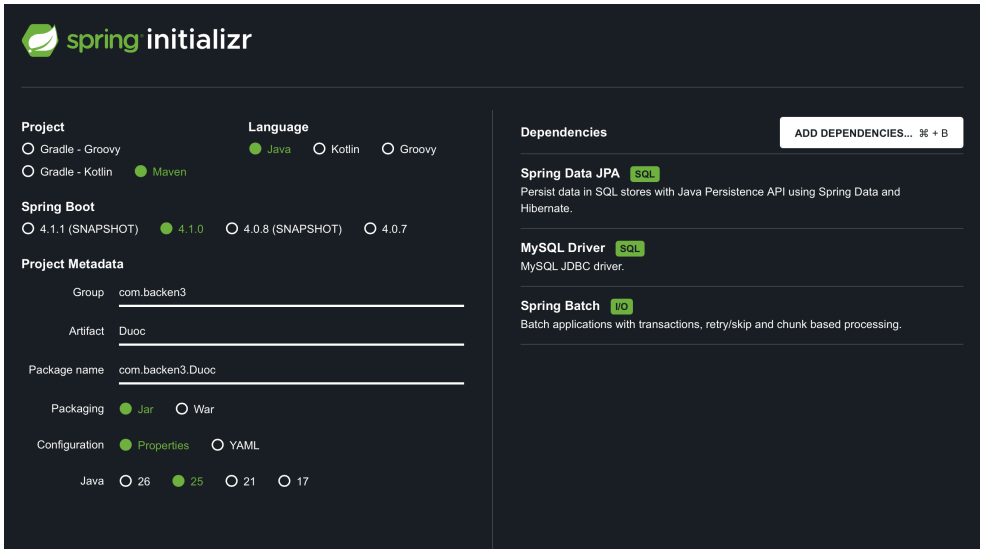
Entrega

- **Código Fuente:** Sube el código del proyecto a un repositorio de GitHub con su propuesta técnica
- **Documentación** (README.md): Describe el objetivo del proyecto, la estructura del código y las instrucciones para ejecutar el proyecto.
- **Evidencia de Ejecución:** Proporciona capturas de pantalla o salidas de consola mostrando la ejecución de cada Job y los resultados generados.

Desarrollo

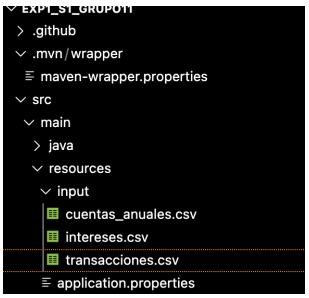
1 - Configurar un Proyecto Spring Batch

Para realizar este proyecto, lo primero que hicimos fue ingresar a spring initialzr y generamos el proyecto con las dependencias de Spring Batch y MySQL Driver



Ya con el proyecto creado, procedí a crear en la carpeta resources una carpeta que se llama input y dentro de ella coloqué los archivos CSV que fueron descargados desde Github, siendo estos los siguientes:

- Cuentas_anuales.csv
- Intereses.csv
- Transacciones.csv



Ahora, antes de comenzar a configurar los Jobs y Steps para leer, procesar y escribir los datos de los archivos CSV, primero configure la conexión a mi base de datos MySQL en el aplication.propperties:

application.properties

src > main > resources > application.properties

```
spring.application.name=Duoc

# Conexion a MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/backendIII?useSSL=false&serverTimezone=Amer
spring.datasource.username=root
spring.datasource.password=Ceci16112010
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

También agregué a mi application.propertis la configuración de mi Spring Batch y la configuración de mi esquema para crear tablas donde se escriben los CSV.

```
# Spring Batch
spring.batch.jdbc.initialize-schema=always

# Ejecuta schema.sql para crear las tablas donde se escriben los CSV.
spring.sql.init.mode=always
```

Posteriormente leemos los archivos CSV para entender su estructura:

Cuentas_anuales.csv, posee los encabezados de cuenta_id, fecha, transacción, monto y descripcion

src > main > resources > input > cuentas_anuales.csv

```
cuenta_id,fecha,transaccion,monto,descripcion
101,2024-01-01,deposito,1000,Ingreso mensual
101,2024-03-15,retiro,-500,Retiro parcial
102,2024-05-22,deposito,1500,Ingreso mensual
103,2024-07-10,deposito,2000,Ingreso mensual
104,2024-09-05,compra,-100,Compra en tienda
105,2024-10-01,deposito,2500,Ingreso extra
106,2024-11-20,deposito,3000,Ingreso mensual
107,2024-12-25,deposito,0,Ingreso navideño
108,2024-12-31,deposito,2000,Ingreso de fin de año
```

Intereses.csv cuenta con los encabezados cuenta_id, nombre, saldo, edad y tipo

src > main > resources > input > intereses.csv

```
1   cuenta_id,nombre,saldo,edad,tipo
2   101,John Doe,5000,30,ahorro
3   102,Jane Smith,8000,25,prestamo
4   103,Bob Johnson,12000,30,prestamo
5   104,Alice Brown,0,45,ahorro
6   105,Charlie Green,7000,35,hipoteca
7   106,John Doe,5000,30,ahorro
8   107,Diana Prince,15000,40,prestamo
9   108,Steve Rogers,10000,80,ahorro
```

Transacciones.csv cuenta con los encabezados id, fecha, monto y tipo

src > main > resources > input > transacciones.csv

```
1   id,fecha,monto,tipo
2   1,2024-01-01,1000,debito
3   2,2024-01-02,1500,credito
4   3,2024-01-03,-200,debito
5   4,2024-01-03,0,debito
6   5,2024-01-04,800,credito
7   6,2024-01-05,700,debito
8   7,2024-01-06,1200,credito
9   8,2024-01-05,700,debito
10  9,2024-01-07,3000,debito
11  10,2024-01-08,1000,credito
```

Esto es importante porque nos permite saber el esquema que va a tener nuestra base de datos MySQL y por ende nos va a permitir crear nuestra nuestros modelos.

Esquema de nuestra base de datos MySQL, de acuerdo a la información que sacamos de los dataset csv.

src > main > resources > schema.sql > ...

```sql
CREATE TABLE IF NOT EXISTS transacciones (
    id BIGINT PRIMARY KEY,
    fecha DATE NOT NULL,
    monto DECIMAL(15, 2) NOT NULL,
    tipo VARCHAR(50) NOT NULL
);

CREATE TABLE IF NOT EXISTS intereses (
    cuenta_id BIGINT PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    saldo DECIMAL(15, 2) NOT NULL,
    edad INT NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    tasa_interes DECIMAL(8, 4) DEFAULT 0,
    interes_calculado DECIMAL(15, 2) DEFAULT 0,
    saldo_final DECIMAL(15, 2) DEFAULT 0
);

CREATE TABLE IF NOT EXISTS cuentas_anuales (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    cuenta_id BIGINT NOT NULL,
    fecha DATE NOT NULL,
    transaccion VARCHAR(50) NOT NULL,
    monto DECIMAL(15, 2) NOT NULL,
    descripcion VARCHAR(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS resumen_transacciones_diarias (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    fecha DATE NOT NULL,
    total_debitos DECIMAL(15, 2) NOT NULL,
    total_creditos DECIMAL(15, 2) NOT NULL,
    total_movimientos INT NOT NULL,
    cantidad_anomalias INT NOT NULL
);

CREATE TABLE IF NOT EXISTS informe_estados_cuenta_anuales (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    cuenta_id BIGINT NOT NULL,
    anio INT NOT NULL,
    total_depositos DECIMAL(15, 2) NOT NULL,
    total_retiros_compras DECIMAL(15, 2) NOT NULL,
    saldo_neto DECIMAL(15, 2) NOT NULL,
    cantidad_movimientos INT NOT NULL
);
```

Ahora creamos nuestro paquete model dentro del proyecto y en su interior creamos las clases CuentasAnuales.java, Interes.java y Transaccion.java, los cuales representan a cada uno de los dataset que fueron entregados en archivo csv.

CuentaAnual.java

src > main > java > com > backen3 > Duoc > model > J CuentaAnual.java > { } com.backen3.Duoc.model

```java
package com.backen3.Duoc.model;

import java.math.BigDecimal;
import java.time.LocalDate;

// Representa una fila del archivo input/cuentas_anuales.csv.
public class CuentaAnual {

    private Long cuentaId;
    private LocalDate fecha;
    private String transaccion;
    private BigDecimal monto;
    private String descripcion;

    public Long getCuentaId() {
        return cuentaId;
    }

    public void setCuentaId(Long cuentaId) {
        this.cuentaId = cuentaId;
    }

    public LocalDate getFecha() {
        return fecha;
    }

    public void setFecha(LocalDate fecha) {
        this.fecha = fecha;
    }

    public String getTransaccion() {
        return transaccion;
    }

    public void setTransaccion(String transaccion) {
        this.transaccion = transaccion;
    }

    public BigDecimal getMonto() {
        return monto;
    }

    public void setMonto(BigDecimal monto) {
        this.monto = monto;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }
}

```

Interes.java

src > main > java > com > backen3 > Duoc > model > J Interes.java > { } com.backen3.Duoc.model

```java
package com.backen3.Duoc.model;

import java.math.BigDecimal;

// Representa una fila del archivo input/intereses.csv.
public class Interes {

    private Long cuentaId;
    private String nombre;
    private BigDecimal saldo;
    private Integer edad;
    private String tipo;

    public Long getCuentaId() {
        return cuentaId;
    }

    public void setCuentaId(Long cuentaId) {
        this.cuentaId = cuentaId;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public BigDecimal getSaldo() {
        return saldo;
    }

    public void setSaldo(BigDecimal saldo) {
        this.saldo = saldo;
    }

    public Integer getEdad() {
        return edad;
    }

    public void setEdad(Integer edad) {
        this.edad = edad;
    }

    public String getTipo() {
        return tipo;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }
}

```

Transaccion.java

src > main > java > com > backen3 > Duoc > model > J Transaccion.java > { } com.backen3.Duoc.model

```java
package com.backen3.Duoc.model;

import java.math.BigDecimal;
import java.time.LocalDate;

// Representa una fila del archivo input/transacciones.csv.
public class Transaccion {

    private Long id;
    private LocalDate fecha;
    private BigDecimal monto;
    private String tipo;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public LocalDate getFecha() {
        return fecha;
    }

    public void setFecha(LocalDate fecha) {
        this.fecha = fecha;
    }

    public BigDecimal getMonto() {
        return monto;
    }

    public void setMonto(BigDecimal monto) {
        this.monto = monto;
    }

    public String getTipo() {
        return tipo;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }
}
```

Ahora creamos la carpeta batch con el objetivo de poder crear nuestra clase CsvBatchConfig.java, aquí haremos toda la lógica de nuestro batch, iniciando por la configuración de nuestros Jobs

```java
@Configuration
public class CsvBatchConfig {

    private static final Logger logger = LoggerFactory.getLogger(CsvBatchConfig.class);

    // Job 1: procesa transacciones diarias y luego genera un resumen para detectar anomalias.
    @Bean
    public Job reporteTransaccionesDiariasJob(JobRepository jobRepository,
            Step transaccionesStep,
            Step resumenTransaccionesDiariasStep) {
        return new JobBuilder(name: "reporteTransaccionesDiariasJob", jobRepository)
                .start(transaccionesStep)
                .next(resumenTransaccionesDiariasStep)
                .build();
    }

    // Ejecuta los tres Jobs en orden cuando arranca la aplicacion.
    @Bean
    public CommandLineRunner ejecutarJobs(JobLauncher jobLauncher,
            Job reporteTransaccionesDiariasJob,
            Job calculoInteresesMensualesJob,
            Job estadosCuentaAnualesJob) {
        return args -> {
            // run.id cambia en cada ejecucion para que Spring Batch no reutilice una ejecucion anterior.
            long runId = System.currentTimeMillis();

            ejecutarJob(jobLauncher, reporteTransaccionesDiariasJob, runId);
            ejecutarJob(jobLauncher, calculoInteresesMensualesJob, runId + 1);
            ejecutarJob(jobLauncher, estadosCuentaAnualesJob, runId + 2);
        };
    }

    // Centraliza el lanzamiento de Jobs para mantener el runner principal y que sea facil de leer.
    private void ejecutarJob(JobLauncher jobLauncher, Job job, long runId) throws Exception {
        JobParameters parametros = new JobParametersBuilder()
                .addLong(key: "run.id", runId)
                .toJobParameters();

        logger.info("Iniciando Job: {}", job.getName());
        jobLauncher.run(job, parametros);
    }

    // Job 2: carga cuentas desde intereses.csv y calcula el saldo final mensual.
    @Bean
    public Job calculoInteresesMensualesJob(JobRepository jobRepository,
            Step interesesStep,
            Step calcularInteresesMensualesStep) {
        return new JobBuilder(name: "calculoInteresesMensualesJob", jobRepository)
                .start(interesesStep)
                .next(calcularInteresesMensualesStep)
                .build();
    }

    // Job 3: carga movimientos anuales y crea un informe agrupado por cuenta.
    @Bean
    public Job estadosCuentaAnualesJob(JobRepository jobRepository,
            Step cuentasAnualesStep,
            Step informeEstadosCuentaAnualesStep) {
        return new JobBuilder(name: "estadosCuentaAnualesJob", jobRepository)
                .start(cuentasAnualesStep)
                .next(informeEstadosCuentaAnualesStep)
                .build();
    }

```

```java
    // Este Step genera un resumen diario desde la tabla transacciones ya cargada.
    @Bean
    public Step resumenTransaccionesDiariasStep(JobRepository jobRepository,
            PlatformTransactionManager transactionManager,
            JdbcTemplate jdbcTemplate) {
        return new StepBuilder(name: "resumenTransaccionesDiariasStep", jobRepository)
                .tasklet((contribution, chunkContext) -> {
                    // Limpiamos el resumen para que refleje la ultima ejecucion del proceso.
                    jdbcTemplate.update("DELETE FROM resumen_transacciones_diarias");

                    // Agrupamos por fecha y calculamos totales, cantidad de movimientos y posibles anomalias.
                    jdbcTemplate.update("""
                            INSERT INTO resumen_transacciones_diarias
                            (fecha, total_debitos, total_creditos, total_movimientos, cantidad_anomalias)
                            SELECT
                                fecha,
                                SUM(CASE WHEN tipo = 'debito' THEN monto ELSE 0 END),
                                SUM(CASE WHEN tipo = 'credito' THEN monto ELSE 0 END),
                                COUNT(*),
                                SUM(CASE WHEN ABS(monto) >= 3000 THEN 1 ELSE 0 END)
                            FROM transacciones
                            GROUP BY fecha
                            """);

                    logger.info("Resumen de transacciones diarias generado correctamente");
                    return RepeatStatus.FINISHED;
                }, transactionManager)
                .build();
    }
```

```java
    // Este Step aplica el interes mensual y actualiza saldo_final en la tabla intereses.
    @Bean
    public Step calcularInteresesMensualesStep(JobRepository jobRepository,
            PlatformTransactionManager transactionManager,
            JdbcTemplate jdbcTemplate) {
        return new StepBuilder(name: "calcularInteresesMensualesStep", jobRepository)
                .tasklet((contribution, chunkContext) -> {
                    // Si la tabla ya existia antes de este cambio, agregamos las columnas calculadas que falten.
                    asegurarColumna(jdbcTemplate, tabla: "intereses", columna: "tasa_interes", definicion: "DECIMAL(8, 4) DEFAULT
                    asegurarColumna(jdbcTemplate, tabla: "intereses", columna: "interes_calculado", definicion: "DECIMAL(15, 2) D
                    asegurarColumna(jdbcTemplate, tabla: "intereses", columna: "saldo_final", definicion: "DECIMAL(15, 2) DEFAULT

                    // Para ahorro se suma 1.5%; para prestamos se descuenta/carga 2.5%; hipoteca usa 2.0%.
                    jdbcTemplate.update("""
                            UPDATE intereses
                            SET
                                tasa_interes = CASE
                                    WHEN tipo = 'ahorro' THEN 0.015
                                    WHEN tipo = 'prestamo' THEN 0.025
                                    WHEN tipo = 'hipoteca' THEN 0.020
                                    ELSE 0
                                END,
                                interes_calculado = ROUND(saldo * CASE
                                    WHEN tipo = 'ahorro' THEN 0.015
                                    WHEN tipo = 'prestamo' THEN 0.025
                                    WHEN tipo = 'hipoteca' THEN 0.020
                                    ELSE 0
                                END, 2),
                                saldo_final = ROUND(saldo + CASE
                                    WHEN tipo = 'ahorro' THEN saldo * 0.015
                                    WHEN tipo = 'prestamo' THEN saldo * 0.025
                                    WHEN tipo = 'hipoteca' THEN saldo * 0.020
                                    ELSE 0
                                END, 2)
                            """);

                    logger.info("Intereses mensuales calculados y saldos finales actualizados");
                    return RepeatStatus.FINISHED;
                }, transactionManager)
                .build();
    }
```

```java
    // Este Step compila los movimientos anuales por cuenta para dejar un informe de auditoria.
    @Bean
    public Step informeEstadosCuentaAnualesStep(JobRepository jobRepository,
            PlatformTransactionManager transactionManager,
            JdbcTemplate jdbcTemplate) {
        return new StepBuilder(name: "informeEstadosCuentaAnualesStep", jobRepository)
                .tasklet((contribution, chunkContext) -> {
                    // Limpiamos el informe anterior para generar una vista anual actualizada.
                    jdbcTemplate.update("DELETE FROM informe_estados_cuenta_anuales");

                    // Agrupamos por cuenta y calculamos ingresos, egresos, cantidad de movimientos y saldo neto anual.
                    jdbcTemplate.update("""
                            INSERT INTO informe_estados_cuenta_anuales
                            (cuenta_id, anio, total_depositos, total_retiros_compras, saldo_neto, cantidad_movimientos)
                            SELECT
                                cuenta_id,
                                YEAR(fecha),
                                SUM(CASE WHEN monto > 0 THEN monto ELSE 0 END),
                                SUM(CASE WHEN monto < 0 THEN ABS(monto) ELSE 0 END),
                                SUM(monto),
                                COUNT(*)
                            FROM cuentas_anuales
                            GROUP BY cuenta_id, YEAR(fecha)
                            """);

                    logger.info("Informe anual de estados de cuenta generado correctamente");
                    return RepeatStatus.FINISHED;
                }, transactionManager)
                .build();
    }
```

Después comenzamos a configurar los 3 steps en orden.

```java
    // paso 1: leer transacciones.csv, procesa cada fila y la escribe en MySQL.
    @Bean
    public Step transaccionesStep(JobRepository jobRepository,
            PlatformTransactionManager transactionManager,
            FlatFileItemReader<Transaccion> transaccionReader,
            ItemProcessor<Transaccion, Transaccion> transaccionProcessor,
            JdbcBatchItemWriter<Transaccion> transaccionWriter) {
        return new StepBuilder(name: "transaccionesStep", jobRepository)
                .<Transaccion, Transaccion>chunk(chunkSize: 5, transactionManager)
                .reader(transaccionReader)
                .processor(transaccionProcessor)
                .writer(transaccionWriter)
                .build();
    }

    // Reader del archivo transacciones.csv: salta el encabezado y asigna columnas al modelo.
    @Bean
    public FlatFileItemReader<Transaccion> transaccionReader() {
        return new FlatFileItemReaderBuilder<Transaccion>()
                .name(name: "transaccionReader")
                .resource(new ClassPathResource("input/transacciones.csv"))
                .linesToSkip(linesToSkip: 1)
                .delimited()
                .names(...names: "id", "fecha", "monto", "tipo")
                .fieldSetMapper(fieldSet -> {
                    // Convertimos manualmente cada columna para controlar tipos como LocalDate y BigDecimal.
                    Transaccion transaccion = new Transaccion();
                    transaccion.setId(fieldSet.readLong(name: "id"));
                    transaccion.setFecha(LocalDate.parse(fieldSet.readString(name: "fecha")));
                    transaccion.setMonto(new BigDecimal(fieldSet.readString(name: "monto")));
                    transaccion.setTipo(fieldSet.readString(name: "tipo"));
                    return transaccion;
                })
                .build();
    }

```

```java
    // Processor de transacciones: valida datos obligatorios, corrige textos y descarta registros incorrectos.
    @Bean
    public ItemProcessor<Transaccion, Transaccion> transaccionProcessor() {
        return transaccion -> {
            // Si el id o la fecha vienen vacios, no se puede guardar una transaccion confiable.
            if (transaccion.getId() == null || transaccion.getFecha() == null) {
                logger.warn("Transaccion descartada por id o fecha vacia: {}", transaccion.getId());
                return null;
            }

            // Si el monto viene vacio, no se puede calcular ni guardar correctamente.
            if (transaccion.getMonto() == null) {
                logger.warn("Transaccion {} descartada por monto vacio", transaccion.getId());
                return null;
            }

            // El monto cero no representa movimiento real, por eso se filtra.
            if (transaccion.getMonto().signum() == 0) {
                logger.warn("Transaccion {} descartada por monto cero", transaccion.getId());
                return null;
            }

            // Se normaliza el tipo para guardar valores consistentes en la base de datos.
            String tipo = normalizarTexto(transaccion.getTipo());
            if (!tipo.equals("debito") && !tipo.equals("credito")) {
                logger.warn("Transaccion {} descartada por tipo invalido: {}", transaccion.getId(), transaccion.getTipo());
                return null;
            }

            transaccion.setTipo(tipo);
            return transaccion;
        };
    }

```

```java
    // Writer de transacciones: inserta cada item procesado en la tabla transacciones.
    @Bean
    public JdbcBatchItemWriter<Transaccion> transaccionWriter(DataSource dataSource) {
        return new JdbcBatchItemWriterBuilder<Transaccion>()
                .dataSource(dataSource)
                .itemSqlParameterSourceProvider(new BeanPropertyItemSqlParameterSourceProvider<>())
                .sql("""
                        INSERT INTO transacciones (id, fecha, monto, tipo)
                        VALUES (:id, :fecha, :monto, :tipo)
                        ON DUPLICATE KEY UPDATE fecha = VALUES(fecha), monto = VALUES(monto), tipo = VALUES(tipo)
                        """)
                .build();
    }

```

```java
    // Paso 2: lee intereses.csv, procesa cada fila y la escribe en MySQL.
    @Bean
    public Step interesesStep(JobRepository jobRepository,
            PlatformTransactionManager transactionManager,
            FlatFileItemReader<Interes> interesReader,
            ItemProcessor<Interes, Interes> interesProcessor,
            JdbcBatchItemWriter<Interes> interesWriter) {
        return new StepBuilder(name: "interesesStep", jobRepository)
                .<Interes, Interes>chunk(chunkSize: 5, transactionManager)
                .reader(interesReader)
                .processor(interesProcessor)
                .writer(interesWriter)
                .build();
    }

```

```java
    // Reader del archivo intereses.csv: mapea cada columna del CSV a la clase Interes.
    @Bean
    public FlatFileItemReader<Interes> interesReader() {
        return new FlatFileItemReaderBuilder<Interes>()
                .name(name: "interesReader")
                .resource(new ClassPathResource("input/intereses.csv"))
                .linesToSkip(linesToSkip: 1)
                .delimited()
                .names(...names: "cuentaId", "nombre", "saldo", "edad", "tipo")
                .fieldSetMapper(fieldSet -> {
                    // Convertimos manualmente cada columna para evitar errores de conversion automatica.
                    Interes interes = new Interes();
                    interes.setCuentaId(fieldSet.readLong(name: "cuentaId"));
                    interes.setNombre(fieldSet.readString(name: "nombre"));
                    interes.setSaldo(new BigDecimal(fieldSet.readString(name: "saldo")));
                    interes.setEdad(fieldSet.readInt(name: "edad"));
                    interes.setTipo(fieldSet.readString(name: "tipo"));
                    return interes;
                })
                .build();
    }
```

```java
    // Processor de intereses: limpia textos, valida saldos/edades y maneja tipos de cuenta permitidos.
    @Bean
    public ItemProcessor<Interes, Interes> interesProcessor() {
        return interes -> {
            // La cuenta es la clave del registro; si falta, no se puede identificar en MySQL.
            if (interes.getCuentaId() == null) {
                logger.warn("Interes descartado porque no tiene cuenta_id");
                return null;
            }

            // Un nombre vacio queda inutil para reportes, por eso se descarta.
            String nombre = limpiarTexto(interes.getNombre());
            if (nombre.isBlank()) {
                logger.warn("Interes de cuenta {} descartado por nombre vacio", interes.getCuentaId());
                return null;
            }

            // Si el saldo viene vacio, el registro queda incompleto para la tabla intereses.
            if (interes.getSaldo() == null) {
                logger.warn("Interes de cuenta {} descartado por saldo vacio", interes.getCuentaId());
                return null;
            }

            // Se descartan saldos negativos porque este archivo representa saldos base de cuentas.
            if (interes.getSaldo().signum() < 0) {
                logger.warn("Interes de cuenta {} descartado por saldo negativo", interes.getCuentaId());
                return null;
            }

            // Se valida la edad para evitar valores imposibles o registros incompletos.
            if (interes.getEdad() == null || interes.getEdad() < 18 || interes.getEdad() > 120) {
                logger.warn("Interes de cuenta {} descartado por edad invalida: {}", interes.getCuentaId(), interes.getEdad());
                return null;
            }

            // Se normaliza el tipo y se aceptan solo los tipos presentes en el negocio del CSV.
            String tipo = normalizarTexto(interes.getTipo());
            if (!tipo.equals("ahorro") && !tipo.equals("prestamo") && !tipo.equals("hipoteca")) {
                logger.warn("Interes de cuenta {} descartado por tipo invalido: {}", interes.getCuentaId(), interes.getTipo());
                return null;
            }
```

```
361
362             interes.setNombre(nombre);
363             interes.setTipo(tipo);
364             return interes;
365         };
366     }
367
368     // Writer de intereses: guarda los datos de cuentas/intereses en MySQL.
369     @Bean
370     public JdbcBatchItemWriter<Interes> interesWriter(DataSource dataSource) {
371         return new JdbcBatchItemWriterBuilder<Interes>()
372                 .dataSource(dataSource)
373                 .itemSqlParameterSourceProvider(new BeanPropertyItemSqlParameterSourceProvider<>())
374                 .sql("""
375                         INSERT INTO intereses (cuenta_id, nombre, saldo, edad, tipo)
376                         VALUES (:cuentaId, :nombre, :saldo, :edad, :tipo)
377                         ON DUPLICATE KEY UPDATE nombre = VALUES(nombre), saldo = VALUES(saldo),
378                         edad = VALUES(edad), tipo = VALUES(tipo)
379                         """)
380                 .build();
381     }
382
383     // paso 3: lee cuentas_anuales.csv, procesa cada fila y la escribe en MySQL.
384     @Bean
385     public Step cuentasAnualesStep(JobRepository jobRepository,
386             PlatformTransactionManager transactionManager,
387             FlatFileItemReader<CuentaAnual> cuentaAnualReader,
388             ItemProcessor<CuentaAnual, CuentaAnual> cuentaAnualProcessor,
389             JdbcBatchItemWriter<CuentaAnual> cuentaAnualWriter) {
390         return new StepBuilder(name: "cuentasAnualesStep", jobRepository)
391                 .<CuentaAnual, CuentaAnual>chunk(chunkSize: 5, transactionManager)
392                 .reader(cuentaAnualReader)
393                 .processor(cuentaAnualProcessor)
394                 .writer(cuentaAnualWriter)
395                 .build();
396     }
397
```

```
398     // Reader del archivo cuentas_anuales.csv: transforma cada linea en un objeto CuentaAnual.
399     @Bean
400     public FlatFileItemReader<CuentaAnual> cuentaAnualReader() {
401         return new FlatFileItemReaderBuilder<CuentaAnual>()
402                 .name(name: "cuentaAnualReader")
403                 .resource(new ClassPathResource("input/cuentas_anuales.csv"))
404                 .linesToSkip(linesToSkip: 1)
405                 .delimited()
406                 .names(...names: "cuentaId", "fecha", "transaccion", "monto", "descripcion")
407                 .fieldSetMapper(fieldSet -> {
408                     // Convertimos manualmente cada linea del CSV al modelo que usara el processor.
409                     CuentaAnual cuentaAnual = new CuentaAnual();
410                     cuentaAnual.setCuentaId(fieldSet.readLong(name: "cuentaId"));
411                     cuentaAnual.setFecha(LocalDate.parse(fieldSet.readString(name: "fecha")));
412                     cuentaAnual.setTransaccion(fieldSet.readString(name: "transaccion"));
413                     cuentaAnual.setMonto(new BigDecimal(fieldSet.readString(name: "monto")));
414                     cuentaAnual.setDescripcion(fieldSet.readString(name: "descripcion"));
415                     return cuentaAnual;
416                 })
417                 .build();
418     }
419
420     // Processor de cuentas anuales: valida movimiento, corrige textos y filtra registros sin monto real.
421     @Bean
422     public ItemProcessor<CuentaAnual, CuentaAnual> cuentaAnualProcessor() {
423         return cuentaAnual -> {
424             // cuenta_id y fecha son campos minimos para relacionar el movimiento anual.
425             if (cuentaAnual.getCuentaId() == null || cuentaAnual.getFecha() == null) {
426                 logger.warn("Cuenta anual descartada por cuenta_id o fecha vacia");
427                 return null;
428             }
429
430             // Si no hay monto, el movimiento queda incompleto y se filtra.
431             if (cuentaAnual.getMonto() == null) {
432                 logger.warn("Cuenta anual de cuenta {} descartada por monto vacio", cuentaAnual.getCuentaId());
433                 return null;
434             }
435
436             // Los movimientos con monto cero se consideran registros sin efecto contable.
437             if (cuentaAnual.getMonto().signum() == 0) {
438                 logger.warn("Cuenta anual de cuenta {} descartada por monto cero", cuentaAnual.getCuentaId());
439                 return null;
440             }
```

```
            // Se normaliza la transaccion y se aceptan solo acciones conocidas.
            String transaccion = normalizarTexto(cuentaAnual.getTransaccion());
            if (!transaccion.equals("deposito") && !transaccion.equals("retiro") && !transaccion.equals("compra")) {
                logger.warn("Cuenta anual de cuenta {} descartada por transaccion invalida: {}",
                        cuentaAnual.getCuentaId(), cuentaAnual.getTransaccion());
                return null;
            }

            // Si no hay descripcion, se corrige con un texto por defecto para cumplir el NOT NULL de la tabla.
            String descripcion = limpiarTexto(cuentaAnual.getDescripcion());
            if (descripcion.isBlank()) {
                descripcion = "Sin descripcion";
            }

            // Se deja consistente el signo del monto segun el tipo de movimiento.
            if ((transaccion.equals("retiro") || transaccion.equals("compra")) && cuentaAnual.getMonto().signum() > 0) {
                cuentaAnual.setMonto(cuentaAnual.getMonto().negate());
            }
            if (transaccion.equals("deposito") && cuentaAnual.getMonto().signum() < 0) {
                cuentaAnual.setMonto(cuentaAnual.getMonto().abs());
            }

            cuentaAnual.setTransaccion(transaccion);
            cuentaAnual.setDescripcion(descripcion);
            return cuentaAnual;
        };
    }

    // Evita repetir trim/toLowerCase cada vez que validamos campos de texto.
    private String normalizarTexto(String texto) {
        return limpiarTexto(texto).toLowerCase();
    }

    // Convierte valores nulos en texto vacio y elimina espacios sobrantes.
    private String limpiarTexto(String texto) {
        if (texto == null) {
            return "";
        }
        return texto.trim();
    }
```

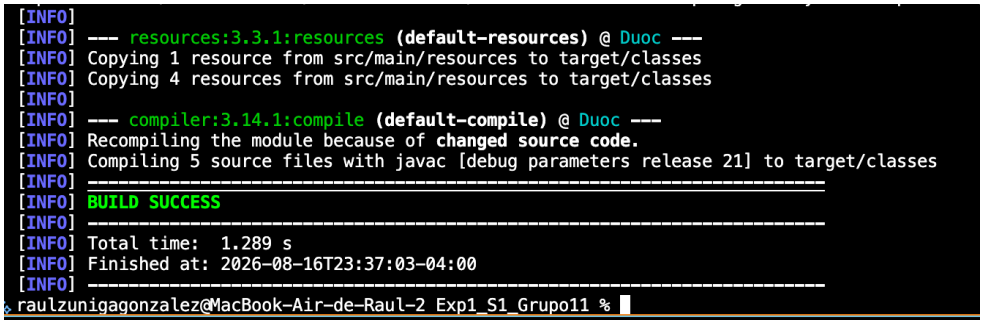
```
    // Agrega una columna solo si no existe; esto evita errores cuando la aplicacion se ejecuta varias veces.
    private void asegurarColumna(JdbcTemplate jdbcTemplate, String tabla, String columna, String definicion) {
        Integer cantidad = jdbcTemplate.queryForObject("""
                SELECT COUNT(*)
                FROM information_schema.COLUMNS
                WHERE TABLE_SCHEMA = DATABASE()
                  AND TABLE_NAME = ?
                  AND COLUMN_NAME = ?
                """, Integer.class, tabla, columna);

        if (cantidad != null && cantidad == 0) {
            jdbcTemplate.execute("ALTER TABLE " + tabla + " ADD COLUMN " + columna + " " + definicion);
        }
    }

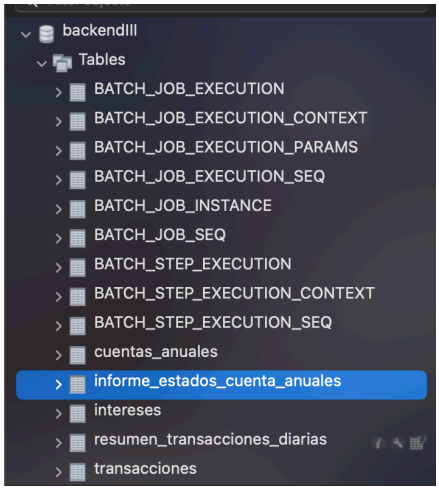
    // Writer de cuentas anuales: inserta los movimientos anuales en MySQL.
    @Bean
    public JdbcBatchItemWriter<CuentaAnual> cuentaAnualWriter(DataSource dataSource) {
        return new JdbcBatchItemWriterBuilder<CuentaAnual>()
                .dataSource(dataSource)
                .itemSqlParameterSourceProvider(new BeanPropertyItemSqlParameterSourceProvider<>())
                .sql("""
                        INSERT INTO cuentas_anuales (cuenta_id, fecha, transaccion, monto, descripcion)
                        VALUES (:cuentaId, :fecha, :transaccion, :monto, :descripcion)
                        """)
                .build();
    }
}
```

2 - Implementar un Procesamiento de Datos

Se ejecuto el proyecto usando ./mvnw spring-boot:run para que compilara. Este se ejecutó de forma correcta y se demoró 1.289 segundos en compilar.



Ahora me voy a mi base d datos de MySQL y me puedo dar cuenta de que mi base de datos BackendIII que presenta una serie de tablas BATCH y al final se encuentran las tablas cuentas_anuales, informe_estados_cuenta_anuales, intereses, resumen_transacciones_diarias y transacciones.



Revisamos el contenido de cada tabla para saber que la información se pasó de forma correcta.

Tabla de cuentas_anuales

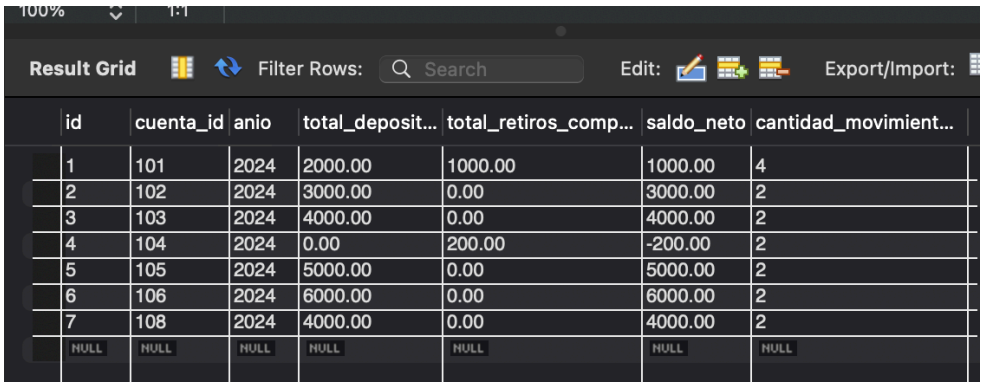


100% 1:1

Result Grid Filter Rows: Search Edit: Export/Import:

id	cuenta_id	fecha	transaccion	monto	descripcion
1	101	2024-01-01	deposito	1000.00	Ingreso mensual
2	101	2024-03-15	retiro	-500.00	Retiro parcial
3	102	2024-05-22	deposito	1500.00	Ingreso mensual
4	103	2024-07-10	deposito	2000.00	Ingreso mensual
5	104	2024-09-05	compra	-100.00	Compra en tienda
6	105	2024-10-01	deposito	2500.00	Ingreso extra
7	106	2024-11-20	deposito	3000.00	Ingreso mensual
8	108	2024-12-31	deposito	2000.00	Ingreso de fin de año
NULL	NULL	NULL	NULL	NULL	NULL

Tabla informe_estados_cuenta_anuales

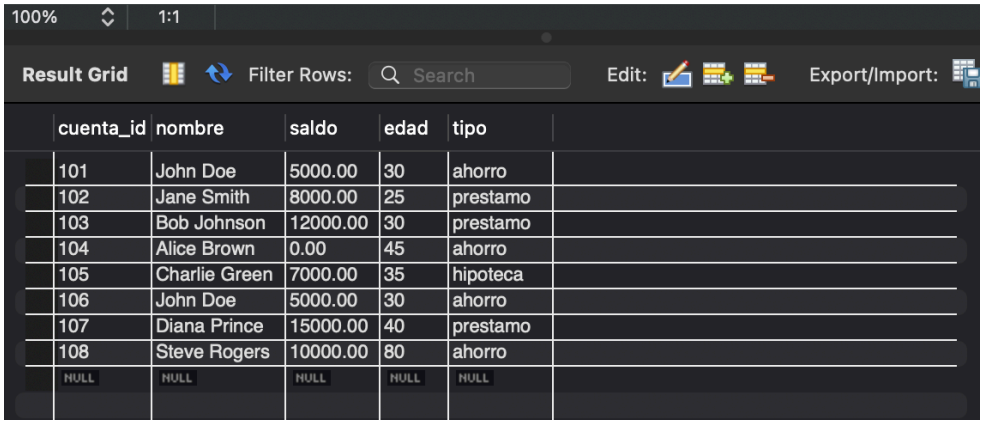


100% 1:1

Result Grid Filter Rows: Search Edit: Export/Import:

id	cuenta_id	anio	total_deposit...	total_retiros_comp...	saldo_neto	cantidad_movimient...
1	101	2024	2000.00	1000.00	1000.00	4
2	102	2024	3000.00	0.00	3000.00	2
3	103	2024	4000.00	0.00	4000.00	2
4	104	2024	0.00	200.00	-200.00	2
5	105	2024	5000.00	0.00	5000.00	2
6	106	2024	6000.00	0.00	6000.00	2
7	108	2024	4000.00	0.00	4000.00	2
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Tabla intereses

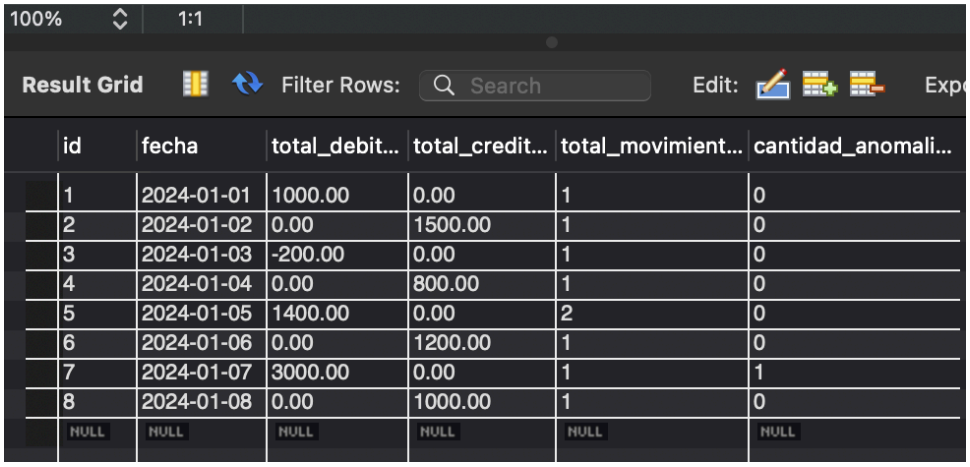


100% 1:1

Result Grid Filter Rows: Search Edit: Export/Import:

cuenta_id	nombre	saldo	edad	tipo
101	John Doe	5000.00	30	ahorro
102	Jane Smith	8000.00	25	prestamo
103	Bob Johnson	12000.00	30	prestamo
104	Alice Brown	0.00	45	ahorro
105	Charlie Green	7000.00	35	hipoteca
106	John Doe	5000.00	30	ahorro
107	Diana Prince	15000.00	40	prestamo
108	Steve Rogers	10000.00	80	ahorro
NULL	NULL	NULL	NULL	NULL

Tabla resumen_transacciones_diarias

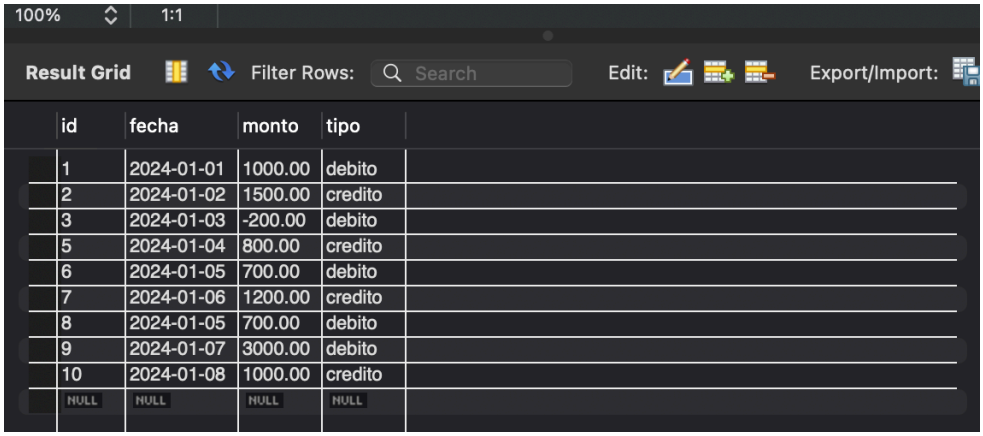


100% 1:1

Result Grid Filter Rows: Search Edit: Expo

id	fecha	total_debit...	total_credit...	total_movimient...	cantidad_anomali...
1	2024-01-01	1000.00	0.00	1	0
2	2024-01-02	0.00	1500.00	1	0
3	2024-01-03	-200.00	0.00	1	0
4	2024-01-04	0.00	800.00	1	0
5	2024-01-05	1400.00	0.00	2	0
6	2024-01-06	0.00	1200.00	1	0
7	2024-01-07	3000.00	0.00	1	1
8	2024-01-08	0.00	1000.00	1	0
NULL	NULL	NULL	NULL	NULL	NULL

Tabla transacciones.



100% 1:1

Result Grid Filter Rows: Search Edit: Export/Import:

id	fecha	monto	tipo
1	2024-01-01	1000.00	debito
2	2024-01-02	1500.00	credito
3	2024-01-03	-200.00	debito
5	2024-01-04	800.00	credito
6	2024-01-05	700.00	debito
7	2024-01-06	1200.00	credito
8	2024-01-05	700.00	debito
9	2024-01-07	3000.00	debito
10	2024-01-08	1000.00	credito
NULL	NULL	NULL	NULL

3 - Manejo de Errores y Excepciones:

El manejo de las excepciones fue controlado en el código en la clase CsvBatchConfig.java, se deja un ejemplo de cómo se trataron.

```java
@Bean
public ItemProcessor<CuentaAnual, CuentaAnual> cuentaAnualProcessor() {
    return cuentaAnual -> {
        // cuenta_id y fecha son campos minimos para relacionar el movimiento anual.
        if (cuentaAnual.getCuentaId() == null || cuentaAnual.getFecha() == null) {
            logger.warn("Cuenta anual descartada por cuenta_id o fecha vacia");
            return null;
        }

        // Si no hay monto, el movimiento queda incompleto y se filtra.
        if (cuentaAnual.getMonto() == null) {
            logger.warn("Cuenta anual de cuenta {} descartada por monto vacio", cuentaAnual.getCuentaId());
            return null;
        }

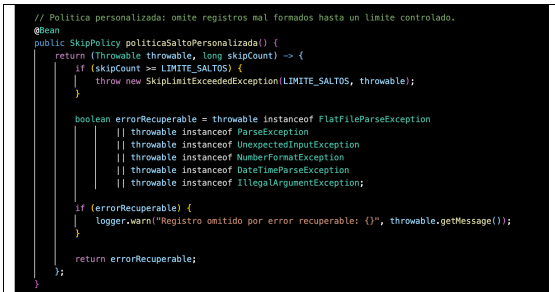
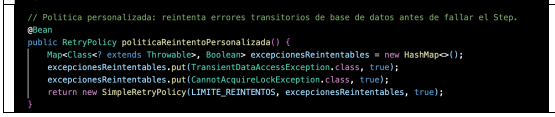
        // Los movimientos con monto cero se consideran registros sin efecto contable.
        if (cuentaAnual.getMonto().signum() == 0) {
            logger.warn("Cuenta anual de cuenta {} descartada por monto cero", cuentaAnual.getCuentaId());
            return null;
        }

        // Se normaliza la transaccion y se aceptan solo acciones conocidas.
        String transaccion = normalizarTexto(cuentaAnual.getTransaccion());
        if (!transaccion.equals("deposito") && !transaccion.equals("retiro") && !transaccion.equals("compra")) {
            logger.warn("Cuenta anual de cuenta {} descartada por transaccion invalida: {}",
                    cuentaAnual.getCuentaId(), cuentaAnual.getTransaccion());
            return null;
        }

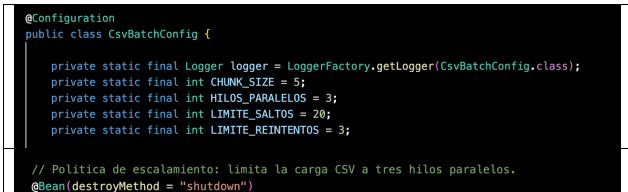
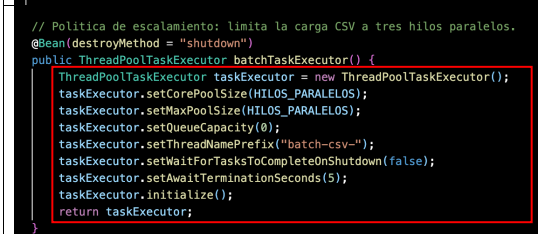
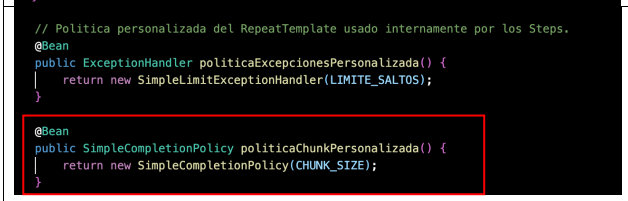
        // Si no hay descripcion, se corrige con un texto por defecto para cumplir el NOT NULL de la tabla.
        String descripcion = limpiarTexto(cuentaAnual.getDescripcion());
        if (descripcion.isBlank()) {
            descripcion = "Sin descripcion";
```

4- Implementar políticas personalizadas y tolerancia a fallos:

Se implementó una política personalizada de tolerancia a fallos

	Se implementó un SkipPolicy, para omitir registros de CSV mal formados hasta un limite.
	Se implementó una RetryPolicy para reintentar errores transitorios de base de datos

5. Implementar políticas de escalamiento:

	De acuerdo a lo solicitado, se implementó en CsvBatchConfig los 5 Chunk y los 3 hilos paralelos
	Se creó una política de escalamiento que va a limitar la carga del CSV en tres hilos paralelos
	Se creó una política personalizada del RepeatTemplate usando internamente los Steps Aquí se configura el chunk de tamaño 5, los cuales serán usados en los tres Steps Principales



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.